

HarvardX - Data Science Capstone - PH125.9x MovieLens Report

April 15, 2026 - December 16, 2026 Session

Jake Coventry

30th April 2026

Contents

1	Introduction	3
2	Data exploration	4
2.1	Data source	4
2.2	Initial inspection of our data	4
2.2.1	Summary statistics	5
2.2.2	Ratings distribution	5
2.2.3	Ratings per movie	6
2.2.4	Ratings per user	7
2.3	Feature engineering	10
2.3.1	Genre	10
2.3.2	Release year	11
2.4	Summary of findings	13
3	Modelling	14
3.1	Evaluation metric	14
3.2	Creating training and validation datasets	14
3.3	Models	15
3.3.1	Global average model	15
3.3.2	Movie effect model b_i	16
3.3.3	User effect model b_u	17
3.3.4	Regularised movie + user effects	18
3.3.5	Feature-enhanced model (genre + release year)	20
4	Results	24
4.1	Final model evaluation on final hold-out test set	24
4.2	Discussion of results	24
4.3	Future work	25

5	Conclusion	26
6	Acknowledgement of generative AI use	27
7	References	27
8	Appendix - Environment	28

1 Introduction

The goal of this project was to build a model that can accurately predict user ratings for movies using the MovieLens 10M dataset collected by GroupLens Research (GroupLens Research, 2009). Model performance was evaluated using Root Mean Squared Error (RMSE), with a target of achieving a value below 0.86490.

Rather than treating recommendation systems in the abstract, this project focused specifically on how users rate films within the MovieLens dataset, containing around 10 million ratings across more than 10,000 movies.

In this project, the model was built sequentially, starting with a simple baseline and gradually adding more complexity. The initial exploratory analysis helped identify patterns in how users rate movies, which then informed the inclusion of movie-specific and user-specific effects, as well as regularisation. Additional features such as genre and release year were also tested to see whether they improve prediction accuracy.

The final model combines these components into a regularised framework that captures the main sources of variation in the data. This results in an RMSE below the required threshold, demonstrating that relatively simple bias-based models can perform well when carefully constructed and validated.

2 Data exploration

Before building the prediction models, the structure of the dataset was explored. The aim of this stage was to understand what information was available, check for any obvious data quality issues, and identify patterns that could influence the modelling approach.

2.1 Data source

The data used for this project came from the MovieLens 10M Dataset, collected by GroupLens Research. It contained user ratings for movies, along with movie-level information such as titles and genres.

2.2 Initial inspection of our data

The main training dataset, *edx*, contained user ratings joined with movie metadata. Each row represents one rating given by a particular user to a particular movie.

The dataset contains 9,000,055 rows and six variables. These variables include the user ID, movie ID, rating, timestamp, movie title, and genres (Table 1). No missing values were identified in any of the columns, so no imputation was needed at this stage.

```
# Check for missing values in each column of the edx training dataset.  
colSums(is.na(edx))
```

```
##      userId      movieId      rating timestamp      title      genres  
##          0           0           0           0           0           0
```

Table 1: *edx* columns and data types extracted using `str()`.

	Datatype
userId	integer
movieId	integer
rating	numeric
timestamp	integer
title	character
genres	character

A preview of the first few rows helped show how the dataset was structured (Table 2). The *timestamp* column records when the rating was submitted in Unix time, while the *title* column includes the movie's release year in brackets. The *genres* column can include more than one genre for a movie, separated by the `|` symbol.

Table 2: first six rows of *edx* data set, extracted using `head()`.

	userId	movieId	rating	timestamp	title	genres
1	1	122	5	838985046	Boomerang (1992)	Comedy Romance
2	1	185	5	838983525	Net, The (1995)	Action Crime Thriller
4	1	292	5	838983421	Outbreak (1995)	Action Drama Sci-Fi Thriller
5	1	316	5	838983392	Stargate (1994)	Action Adventure Sci-Fi
6	1	329	5	838983392	Star Trek: Generations (1994)	Action Adventure Drama Sci-Fi
7	1	355	5	838984474	Flintstones, The (1994)	Children Comedy Fantasy

These variables were useful for deciding which features to explore further. In particular, genre and release year seemed likely to contain additional information that could help explain differences in ratings, and were used during feature engineering 2.3.

2.2.1 Summary statistics

Basic summary statistics were calculated to understand the size and structure of the dataset. These include the number of unique users and movies, as well as the distribution of ratings.

```
# Calculate the number of unique users and movies in the dataset.
edx %>%
  summarize(
    Unique_users=n_distinct(userId),
    Unique_movies=n_distinct(movieId),
    Total_ratings=n()
  )
```

```
##   Unique_users Unique_movies Total_ratings
## 1           69878         10677      9000055
```

The dataset contained 69,878 users, 10,677 movies, and just over 9 million ratings. On average, this is around 840 ratings per movie, although this average does not show the full picture because some movies are rated far more often than others.

Even though the dataset is large, it is still very sparse. If every user had rated every movie, there would be around 746 million possible ratings. The actual number of ratings is much smaller than this, which means that most users have only rated a small proportion of the available movies.

This sparsity is important for the modelling stage. Since many user–movie combinations are missing, the model needs to make predictions using limited information. This helped motivate the use of movie and user bias terms, as well as regularisation later in modelling (section 3).

2.2.2 Ratings distribution

Ratings range from 0.5 to 5.0, increasing in steps of 0.5. The average rating is about 3.5, which is useful because it can be used as a simple baseline prediction before adding more detailed model components.

```
summary(edx$rating) # Summarise the distribution of ratings.
```

```
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.500   3.000   4.000   3.512   4.000   5.000
```

```
sort(unique(edx$rating)) # Sort so they are in numerical order when presenting in markdown
```

```
## [1] 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

Figure (1) shows that the ratings are not evenly distributed across the scale. Most ratings are between 3 and 5, with fewer ratings at the lower end. This means the data is skewed toward higher ratings and is an example of positivity bias.

This pattern makes sense for a movie recommendation dataset. Users are more likely to watch and rate movies they are already interested in, so the ratings do not represent a completely random sample of all possible user–movie combinations.

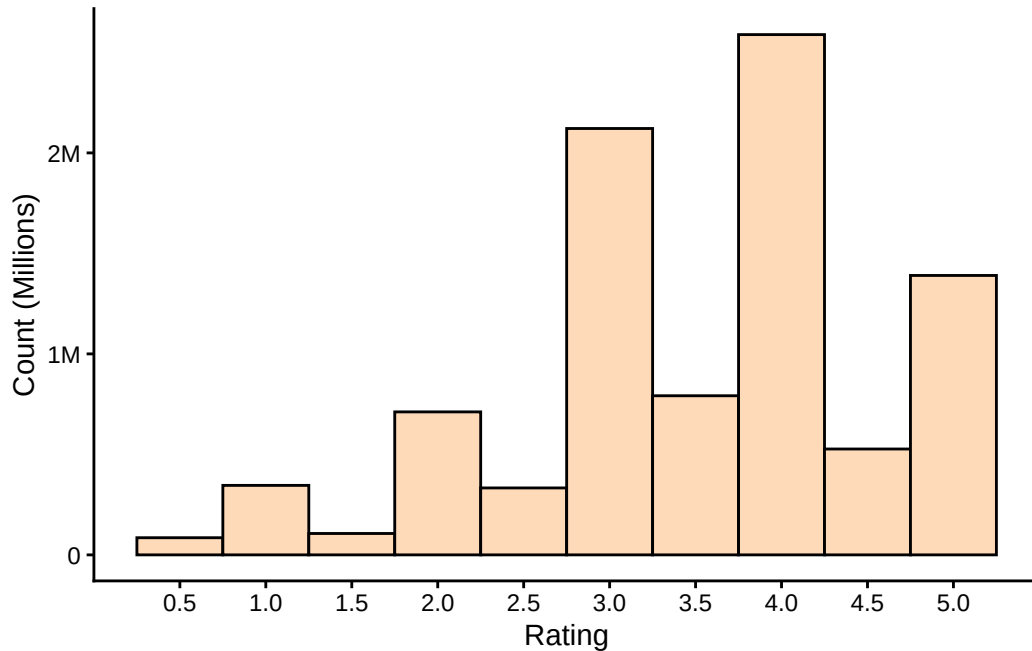


Figure 1: Histogram showing distribution of movie ratings

The skew in the ratings also matters for modelling. A model that only predicts the global average will miss important differences between movies and users. It may underestimate movies that are usually rated highly and overestimate movies that are usually rated poorly. This helped motivate the use of movie-specific and user-specific effects later in the modelling section.

It is also worth noting that ratings are only given in half-star increments. Although the models treat ratings as numeric values, the original rating scale is discrete, which should be kept in mind when interpreting RMSE.

2.2.3 Ratings per movie

The number of ratings per movie varied significantly across the dataset, from movies with only 1 rating to movies with over 31,000 ratings.

```
# Calculate the number of ratings per movie.
ratings_per_movie <- edx %>%
  count(movieId, title, sort = TRUE)

# summarise the distribution of ratings per movie.
summary(ratings_per_movie$n)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       1.0   30.0   122.0   842.9  565.0 31362.0
```

The summary statistics show a clear imbalance. The median movie has 122 ratings, while the mean is much higher at about 843 ratings. This difference shows that the distribution is strongly right-skewed: most movies have relatively few ratings, while a small number of very popular movies have a very large number.

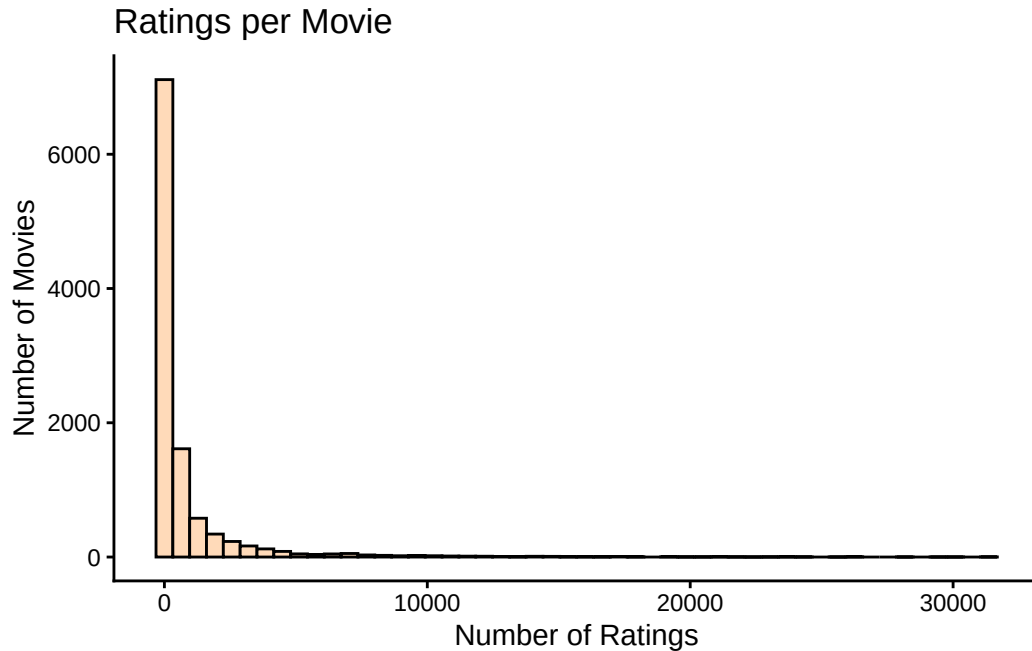


Figure 2: Initial histogram showing distribution of how many ratings each movie has received, with unscaled x axis

The original-scale histogram (Fig 2) showed this problem visually. Because a few movies have extremely high numbers of ratings, most of the other movies are compressed into the left side of the plot, making the distribution difficult to interpret.

The ratings-per-movie histogram was re-plotted on a log scale (Fig 3) to make the distribution of the number of ratings per movie more clear.

This was important for the modelling stage because movie averages are not equally reliable. A movie with thousands of ratings gives a much more stable estimate of its average rating than a movie with only one or two ratings. Without adjustment, the model could give too much importance to movies with very few ratings and overfit to random noise.

For this reason, the movie-level analysis helped motivate the use of regularisation. Regularisation shrinks movie effects toward the global average, especially when they are based on fewer ratings, which should make the model more stable on unseen data.

2.2.4 Ratings per user

```
# Calculate the number of ratings per user.
ratings_per_user <- edx %>%
  count(userId, sort = TRUE)

# summarise the distribution of ratings per user.
summary(ratings_per_user$n)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      10.0   32.0   62.0   128.8  141.0 6616.0
```

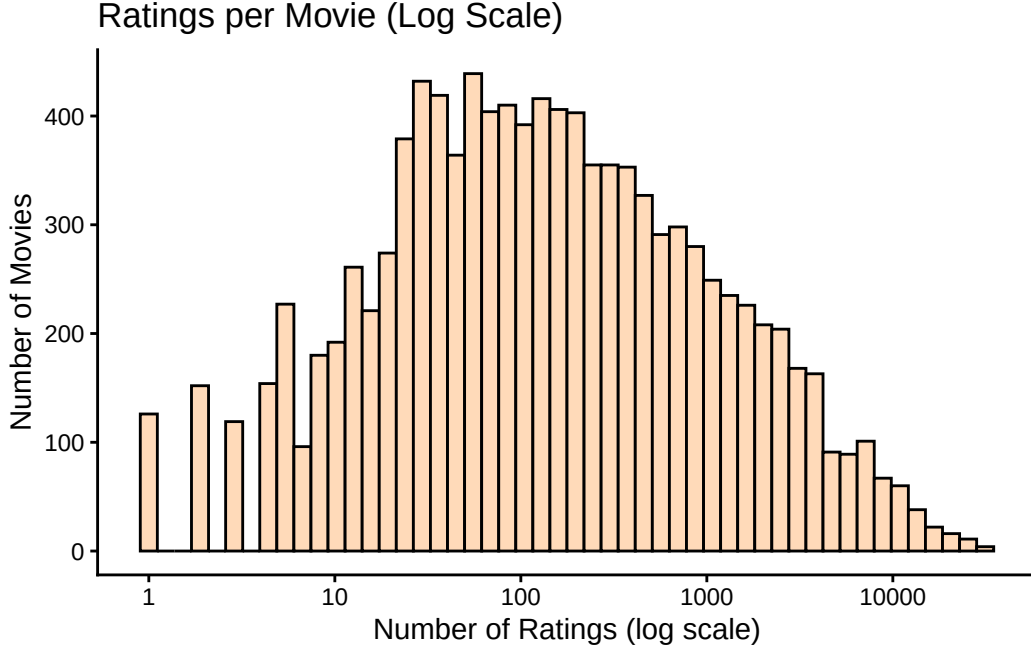


Figure 3: Histogram showing distribution of how many ratings each movie has received, with x axis log scaled

The number of ratings per user ranges from 10 to 6,616, showing that users vary a lot in how active they are. The median user has rated 62 movies, while the mean is about 129 ratings. Since the mean is much higher than the median, this again suggests a right-skewed distribution.

As with the movie-level ratings, the original-scale histogram was difficult to interpret because a small number of very active users dominated the scale. Using a logarithmic x-axis made the pattern clearer. The log-scale histogram (Fig 4) shows that most users rated a relatively small number of movies, while a smaller group of users rated hundreds or even thousands.

This matters for the model because users differ not only in how many ratings they give, but also in how they use the rating scale. Some users tend to give higher ratings overall (Fig 5), while others are more critical. This supports the inclusion of user-specific effects in the model.

It also showed why regularisation is useful for user effects. A user with many ratings provides a more reliable estimate of their general rating behaviour, while a user with only a small number of ratings gives less information. Regularisation helps by shrinking less reliable user effects toward the global average, reducing the chance that the model overfits to sparse user data.

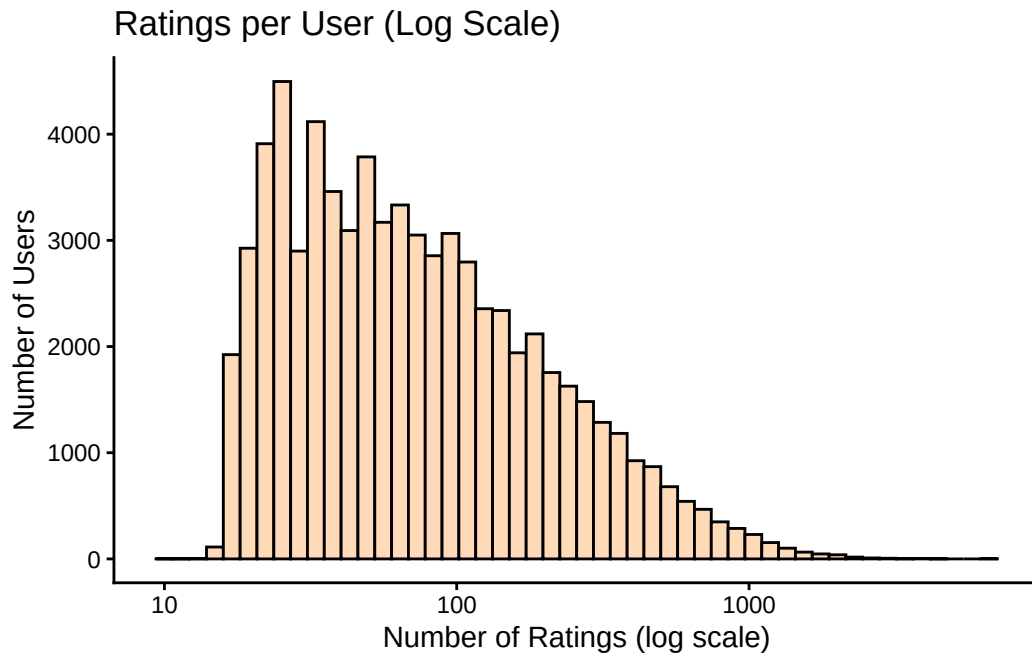


Figure 4: histogram showing distribution of how many ratings each user provided, with x axis log scaled

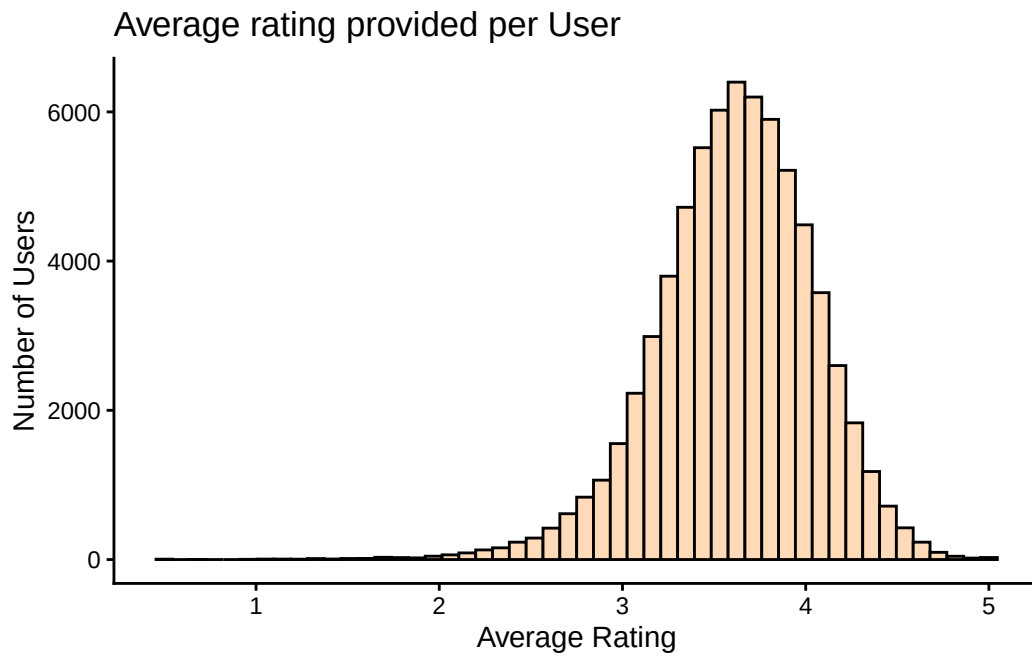


Figure 5: histogram showing average rating that each user provided

2.3 Feature engineering

After the initial data exploration, a small number of additional features were created from the movie meta-data. The aim was to see whether information such as genre or release year could help explain rating patterns beyond the user ID and movie ID.

2.3.1 Genre

The genres variable can contain more than one genre for each movie, with genres separated by the | symbol. For example, a movie might be listed as *Action/Adventure/Sci-Fi*.

To keep the model simple and easier to interpret, only the first listed genre was used as the movie’s main genre. This is a limitation, because it means some information is lost, but it provided a straightforward way to test whether genre had any predictive value.

Table 3: genres extracted from genres column, ordered by average rating.

main_genre	avg_rating	sd_rating	n
Film-Noir	4.145943	0.8296371	15811
Crime	3.871154	0.9400519	529521
Documentary	3.787256	1.0015611	80966
Mystery	3.700779	0.9540623	30536
War	3.672429	0.8685057	2314
Drama	3.666475	0.9958351	1741668
(no genres listed)	3.642857	1.1073349	7
Musical	3.642031	1.1139611	16264
Adventure	3.564558	1.0384934	753650
Animation	3.549188	1.0404264	218123
Western	3.535915	0.9813449	15300
Thriller	3.529973	0.9670796	94718
Comedy	3.453823	1.0747385	2437260
Sci-Fi	3.433587	1.0869653	50254
Action	3.421405	1.0665828	2560545
Fantasy	3.309969	1.0289570	26080
Romance	3.280374	1.0626416	12733
Children	3.247557	1.1519006	181217
Horror	3.033500	1.2009325	233074
IMAX	2.321429	1.2342413	14

(Table 3) shows the average rating for each extracted main genre. There are clear differences between genres. For example, Film-Noir, Crime, and Documentary have relatively high average ratings, while Horror and IMAX have lower average ratings.

This suggests that genre may help explain some variation in ratings. One possible reason is that different genres attract different audiences, and some genres may be rated more consistently than others. However, genre was treated as a supporting feature rather than a main driver of ratings, because movie-specific effects are likely to capture much more detailed information about each individual film.

There are also some limitations to this feature. Using only the first listed genre ignores the fact that many movies belong to several categories. In addition, genre may overlap with other factors, such as movie popularity or release year. For this reason, genre should not be interpreted as directly causing higher or lower ratings. Instead, it was included as an additional signal that might slightly improve prediction accuracy.

2.3.2 Release year

Movie release year was extracted from the *title* variable, where it appears in brackets at the end of each movie title. This allowed me to check whether ratings varied depending on when a movie was released.

```
# Extract the release year from the movie title.
edx <- edx %>%
  mutate(
    year = str_extract(title, "\\([0-9]{4}\\)") %>% # using reg ex to extract release year
    str_remove_all("\\(|\\)") %>%
    as.numeric()
  )

# summarise the distribution of release years.
summary(edx$year)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1915   1987   1994    1990   1998    2008
```

The movies in the dataset were released between 1915 and 2008, with the average release year around 1990. (Fig 6) shows that many of the movies are from the 1990s and early 2000s, which means the dataset is not evenly spread across all release years.

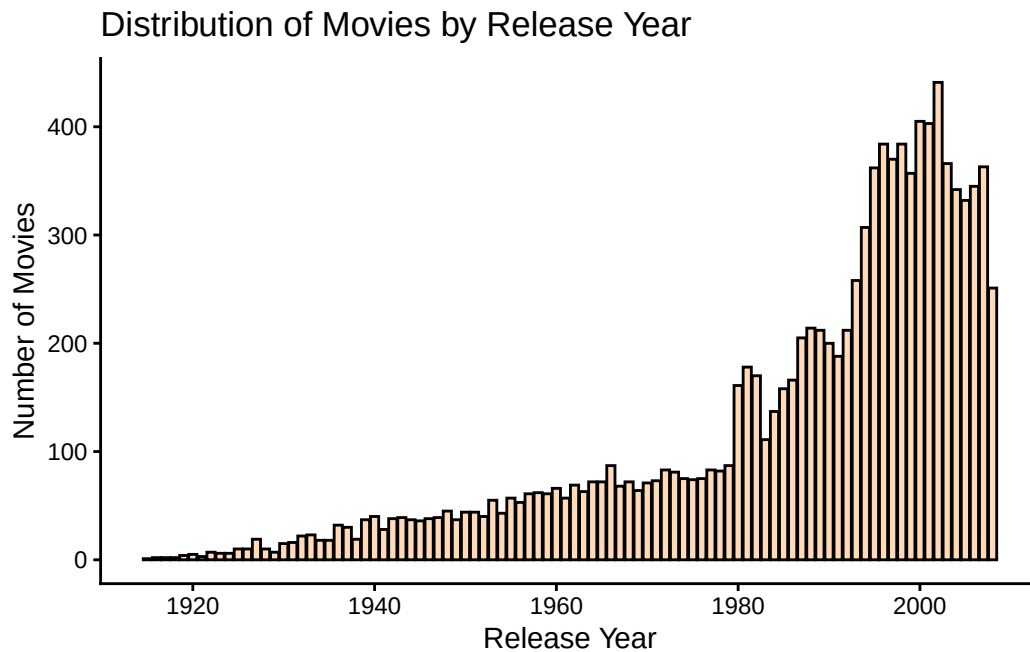


Figure 6: histogram showing distribution of movies by release year

Two related patterns were explored: the number of ratings by release year and the average rating by release year. The number of ratings varies a lot across years, with more ratings generally appearing for newer decades. This is likely because the dataset contains more films from recent years, this can be seen clearly when overlaying the number of movies released with the ratings volume (Fig 7). Evidently, there are more ratings when there are more movies. Additionally, these films may also have been more familiar to users when the ratings were collected.

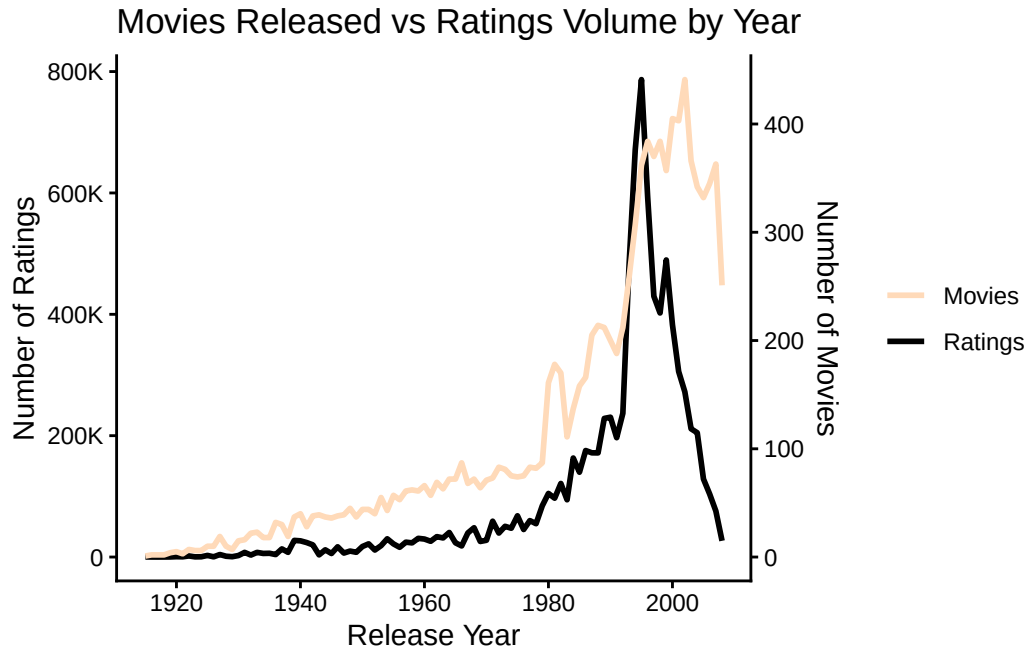


Figure 7: line chart showing number of ratings volume by movie release year

The average rating by release year showed a different pattern (Fig 8). Older films, especially those from around the mid-20th century, appeared to have higher average ratings than many newer films. One possible explanation is survivorship bias. Older films that remain in the dataset and continue to be rated are more likely to be well-known or highly regarded, while newer years contain a wider mix of films.

This suggested that release year could provide some extra predictive information. However, it was treated as a supporting feature rather than a direct explanation for ratings, because release year may also be related to other factors such as genre, popularity, or which films were included in the dataset.

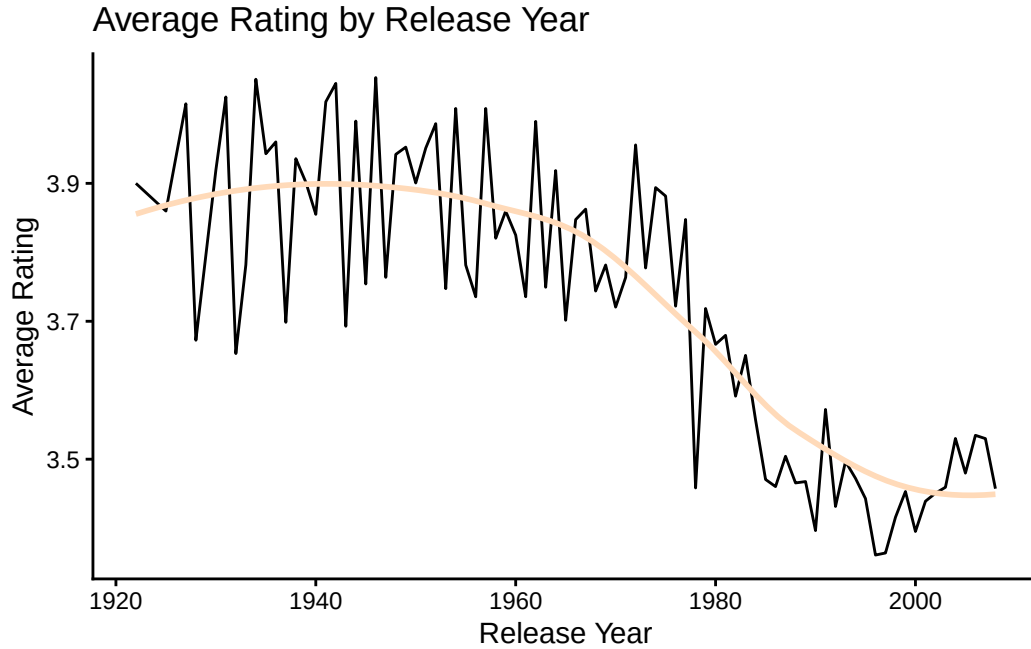


Figure 8: line chart showing average rating per release year

2.4 Summary of findings

The exploratory analysis highlighted several patterns that were important for the modelling stage. First, the ratings were skewed toward higher values, with most ratings falling between 3 and 5. This meant that a simple average rating could be useful as a baseline, but it would not be enough to explain the differences between users and movies.

The analysis also showed strong long-tail patterns in both movie popularity and user activity. Some movies had thousands of ratings, while many had only a small number. The same was true for users, where a small group of users rated many movies, but most users rated far fewer. This was important because estimates based on only a few ratings are less reliable and can make the model more likely to overfit.

Genre and release year also showed some variation in average ratings. This suggested that they might add useful information to the model, despite the expectation that their effect would be smaller than the movie and user effects.

Overall, these findings supported a step-by-step modelling approach. Starting with a global average baseline, then added movie effects and user effects, before applying regularisation to improve stability. Finally, genre and release year were tested to see whether they provided a measurable improvement in RMSE.

3 Modelling

3.1 Evaluation metric

To evaluate model performance, Root Mean Squared Error (RMSE) was used as the primary metric. RMSE measures how far predictions are from the true ratings on average. In practical terms, it can be interpreted as the typical error in predicted ratings.

RMSE is defined as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE is well suited to this rating prediction problem because it is expressed in the same units as the original ratings. This makes it easy to interpret as the typical prediction error in stars. In addition, because errors are squared before averaging, RMSE penalises larger prediction errors more heavily than smaller ones. This is useful in a recommendation context, where large errors may lead to poor recommendations.

However, RMSE also has limitations. It is sensitive to outliers and does not directly measure ranking quality or user satisfaction, both of which are important in real-world recommender systems. Despite these limitations, RMSE is the required evaluation metric for this capstone project and provides a consistent way to compare model performance.

In this analysis, RMSE is used to compare all candidate models. Lower RMSE values indicate better predictive performance. Importantly, RMSE is calculated on validation and holdout data that are not used during model training, reducing the risk of overfitting and data leakage.

3.2 Creating training and validation datasets

A separate validation set was created from the *edx* dataset so that the candidate models could be compared on data not used during training. Around 10% of the observations were placed in the validation set, with the remaining 90% used for training.

The validation set was used for model comparison and for tuning the regularisation parameter λ . The separate *final_holdout_test* dataset was kept aside and was not used during model development or tuning. This meant that it could be used later as a final check of model performance.

```
# Create a reproducible train/validation split from the edx dataset.
set.seed(2, sample.kind = "Rounding")

# Select 10% of edx as a validation set while preserving the rating distribution.
test_index <- createDataPartition(
  y = edx$rating,
  times = 1,
  p = 0.1,
  list = FALSE
)

# Use 90% of edx for training and 10% for validation.
train_set <- edx[-test_index, ]
validation_set <- edx[test_index, ]
```

After making the split, *userId* and *movieId* used in the validation set were included in the training set. This was necessary because the models estimate user and movie effects from the training data. If a user or movie appeared only in the validation set, the model would not have an estimated effect for it.

```
# Ensure all users and movies in validation are in training
validation_set <- validation_set %>%
  semi_join(train_set, by = "movieId") %>%
  semi_join(train_set, by = "userId")

# Add any removed validation rows back into the training set so
# that no valid observations are discarded from model development.
removed <- anti_join(edx[test_index, ], validation_set)
train_set <- rbind(train_set, removed)

# Remove temporary objects to keep the workspace clean.
rm(test_index, removed)
```

Any rows removed from the validation set during this check were added back into the training set. This allowed the validation set to be used fairly while also keeping the training data as complete as possible.

3.3 Models

3.3.1 Global average model

A simple baseline model using the average rating from the training set was used as the baseline. This model predicts the same rating for every user and movie, so it does not take into account which movie is being rated or which user is giving the rating.

The model can be written as:

$$\hat{y}_{ui} = \mu$$

where μ is the mean rating in the training data.

This baseline is useful because it gives a starting point for comparison. Any more complex model should improve on this RMSE to justify the extra complexity.

```
# Compute global average rating using training data only
mu <- mean(train_set$rating)
mu
```

```
## [1] 3.512406
```

The mean rating in the training set was 3.512406, so the baseline model predicts this value for every observation in the validation set.

```
# Predict the same value for every observation in validation set
baseline_preds <- rep(mu, nrow(validation_set))

# Compute RMSE
baseline_rmse <- RMSE(validation_set$rating, baseline_preds)
baseline_rmse
```

```
## [1] 1.06031
```

The baseline model produced a validation RMSE of 1.06031.

This result shows the limitation of using only the global average. Since every prediction is the same, the model will tend to overestimate low ratings and underestimate high ratings. It also cannot account for the fact that some movies are generally rated higher than others, or that some users tend to give higher or lower ratings overall.

Because of this, the baseline model was mainly used as a reference point. The next models add movie-specific and user-specific effects to capture more of the structure seen in the exploratory analysis.

3.3.2 Movie effect model b_i

The first improvement made to the baseline model was to add a movie-specific effect. This was based on the earlier finding that movies are not rated equally (section 2.2.3). Some movies tend to receive higher ratings than average, while others tend to receive lower ratings.

The model can be written as:

$$\hat{y}_{ui} = \mu + b_i$$

where

- μ is the global average rating
- b_i is the movie-specific effect for movie i .

The movie effect measures how much a movie's average rating differs from the overall mean.

```
# estimate the movie-specific effect (b_i). This measures how much each
# movie's average rating deviates from the global mean (mu).

movie_effects <- train_set %>%
  group_by(movieId) %>%
  summarize(
    b_i = mean(rating - mu),
    .groups = "drop"
  )

# generate predictions on the validation set.
pred_movie <- validation_set %>%
  left_join(movie_effects, by = "movieId") %>%
  mutate(
    b_i = ifelse(is.na(b_i), 0, b_i), # join the movie effects and compute
    # predicted ratings as prediction = global mean + movie effect
    pred = mu + b_i
  )

# compute RMSE to evaluate model performance on unseen (validation) data.
rmse_movie <- RMSE(validation_set$rating, pred_movie$pred)

# print value in rmd
rmse_movie
```

```
## [1] 0.9436226
```


The movie effect model produced a validation RMSE of 0.9436226, which was a clear improvement over the baseline RMSE of 1.06031 (Table 4).

This shows that movie-level differences explain a large amount of variation in the ratings. A positive b_i means that a movie is rated above average, while a negative b_i means that it is rated below average.

However, this approach still has a limitation. The movie effect is more reliable for movies with many ratings than for movies with only a few. A movie with very few ratings could have an unusually high or low estimated effect just by chance. This issue is addressed later using regularisation.

Table 4: Baseline and movie effect RMSE comparison

model	RMSE
Baseline	1.0603099
Movie Effect	0.9436226

3.3.3 User effect model b_u

The movie effect model improved performance by accounting for differences between movies, but it still assumed that all users rate movies in the same way. This is not realistic, because some users tend to give high ratings more often, while others are generally more critical.

To account for this, a user-specific effect was added. The model becomes:

$$\hat{y}_{ui} = \mu + b_i + b_u$$

where:

- μ is the global average rating
- b_i is the movie-specific effect
- b_u is the user-specific effect

The user effect b_u measures how much a user tends to rate above or below what would be expected after accounting for the movie effect.

```
# Estimate the user-specific effect (b_u).
user_effects <- train_set %>%
  left_join(movie_effects, by = "movieId") %>%
  group_by(userId) %>%
  summarize(
    b_u = mean(rating - mu - b_i),
    .groups = "drop"
  )

# Generate predictions on the validation set.
pred_user <- validation_set %>%
  left_join(movie_effects, by = "movieId") %>%
  left_join(user_effects, by = "userId") %>%
  mutate(
    b_i = ifelse(is.na(b_i), 0, b_i), # handle unseen movies
    b_u = ifelse(is.na(b_u), 0, b_u), # handle unseen users
    pred = mu + b_i + b_u
  )
```

```
# Compute RMSE to evaluate performance.
rmse_user <- RMSE(validation_set$rating, pred_user$pred)

rmse_user
```

```
## [1] 0.8663835
```

Adding user effects reduced the validation RMSE to 0.8663835, which was a large improvement over the movie-only model (Table 5).

This shows that user behaviour is an important part of the rating pattern. A positive b_u means that a user tends to give higher ratings than expected, while a negative b_u means that they tend to give lower ratings.

Table 5: Baseline, movie effect, and user effect RMSE comparison

model	RMSE
Baseline	1.0603099
Movie Effect	0.9436226
Movie + User Effect	0.8663835

This step also introduced a basic form of personalisation. Instead of only asking whether a movie is generally rated highly, the model also adjusts for how generous or critical each user tends to be.

However, the same issue seen with movie effects also applies here. User effects are more reliable when a user has rated many movies, and less reliable when a user has only a small number of ratings. This is one of the reasons regularisation is added in the next model.

3.3.4 Regularised movie + user effects

Although adding movie and user effects improved the model, it also created a possible overfitting issue. This was especially relevant for movies or users with only a small number of ratings.

In the previous model, the movie and user effects were calculated using simple averages. This works well when there are many ratings, but it can be unreliable when there are only a few observations. For example, if a movie has only one or two ratings, its estimated movie effect could be very high or very low just because of random variation rather than because the movie is genuinely rated that way by most users.

To reduce this problem, regularisation was introduced. The purpose of regularisation is to shrink the estimated movie and user effects toward zero, meaning that uncertain effects are pulled closer to the global average. This is useful because effects based on fewer ratings should not be trusted as much as effects based on many ratings.

The model remains:

$$\hat{y}_{ui} = \mu + b_i + b_u \quad (1)$$

The difference is in how the effects are estimated. Instead of using simple averages, a regularisation term λ is added to the denominator:

$$b_i = \frac{\sum(r_{u,i} - \mu)}{n_i + \lambda}, \quad b_u = \frac{\sum(r_{u,i} - \mu - b_i)}{n_u + \lambda}$$

where:

- μ is the global average rating
- b_i is the movie-specific effect
- b_u is the user-specific effect
- r_{ui} is the rating given by user u to movie i
- n_i is the number of ratings for movie i
- n_u is the number of ratings for user u
- λ is the regularisation parameter

As λ increases, the denominator becomes larger, so the estimated effects become smaller in magnitude. This means that the model becomes more conservative, especially for movies and users with limited data.

This creates a trade-off:

- Small λ : The model may overfit by giving too much importance to noisy estimates.
- Large λ : The model may shrink the effects too much and fail to capture genuine differences between users and movies

For this reason, λ was tuned using the validation set.

3.3.4.1 Lambda tuning To choose a suitable value for λ , a range of possible values were tested, and the validation RMSE was calculated for each one. The value that produced the lowest validation RMSE was selected.

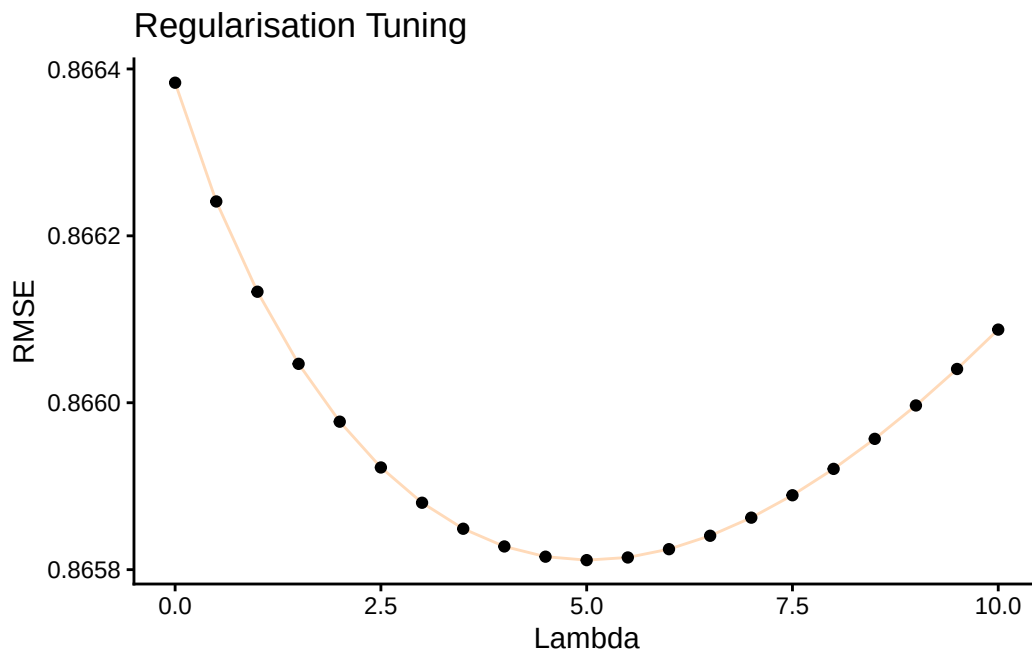


Figure 9: regularisation tuning: RMSE by lambda

```
# Identify the lambda value that minimizes validation RMSE.
best_lambda <- results %>%
  slice_min(rmse) %>%
  pull(lambda)
```

```
# Extract the corresponding minimum RMSE.
best_rmse <- min(results$rmse)

# Display the optimal lambda and its RMSE.
best_lambda
```

```
## [1] 5
```

```
best_rmse
```

```
## [1] 0.8658114
```

The best value of λ was 5, giving a validation RMSE of 0.8658114. This was a slight improvement over the unregularised movie and user effect model, which had an RMSE of 0.8663835.

Although the improvement is small, it is still useful because it shows that regularisation helped the model generalise slightly better. This makes sense given the earlier exploratory analysis, where both movie ratings and user activity showed strong long-tail patterns. Some movies and users had many observations, while others had very few, so shrinking less reliable estimates helped reduce noise.

Table 6 shows that the RMSE changes only slightly across the tested lambda values, but the lowest point occurs when λ is 5. This suggests that a moderate amount of regularisation was enough to improve stability without removing too much useful information from the movie and user effects.

Table 6: Baseline, movie effect, and user effect and regularisation RMSE comparison

model	RMSE
Baseline	1.0603099
Movie Effect	0.9436226
Movie + User Effect	0.8663835
Regularised Model (Movie + User Effect)	0.8658114

Overall, regularisation provided a small but meaningful improvement. The main benefit was not that it changed the model structure, but that it made the estimated movie and user effects more stable. This was particularly important for users and movies with fewer ratings, where simple average-based effects were more likely to be unreliable.

3.3.5 Feature-enhanced model (genre + release year)

After building the regularised movie and user effect model, two engineered features from the exploratory analysis were added: main genre and release year. These were tested to determine whether they provided predictive information beyond the regularised movie and user effects. Although the improvement was modest, the validation results showed a measurable gain, so these terms were included in the final model.

These features were chosen because the exploratory analysis showed differences in average ratings across genres and release years. However, their effects were expected to be smaller than the movie and user effects, since movie ID already captures much of the variation between individual films.

The model was extended as follows:

$$\hat{y}_{ui} = \mu + b_i + b_u + b_g + b_y$$

where:

- μ is the global average rating
- b_i is the movie-specific effect
- b_u is the user-specific effect
- b_g is the genre effect
- b_y is the release year effect

The genre effect was included to account for broad differences in how different types of movies are rated. For example, the exploratory analysis showed that some genres had higher average ratings than others. The release year effect was included because ratings also appeared to vary depending on when a movie was released. Older films in the dataset may be affected by survivorship bias, since older movies that remain well known are often those that were more highly regarded.

As in the previous model, regularisation was used when estimating these effects. This was important because some genres or release years had far fewer observations than others, which could lead to too much weight being placed on small or potentially noisy groups. For simplicity and consistency, the same lambda value selected during the regularised movie and user effect model was reused for the genre and release year effects, rather than tuning a separate lambda for each additional feature.

One limitation of this approach is that only the first listed genre was used for each movie. This made the modelling process simpler, but it also meant that some information was lost. For example, a film listed as *Action/Adventure/Sci-Fi* was treated only as *Action*, even though the other genres may also help explain how users rate it.

```
# Add engineered features to both training and validation sets.
```

```
train_set <- train_set %>%
  mutate(
    genre = str_split_fixed(genres, "\\|", 2)[,1],
    year = str_extract(title, "\\([0-9]{4}\\)") %>%
      str_remove_all("\\(|\\)") %>%
      as.numeric()
  )
```

```
validation_set <- validation_set %>%
  mutate(
    genre = str_split_fixed(genres, "\\|", 2)[,1],
    year = str_extract(title, "\\([0-9]{4}\\)") %>%
      str_remove_all("\\(|\\)") %>%
      as.numeric()
  )
```

```
# Use the lambda value selected during validation tuning.
```

```
lambda <- best_lambda
```

```
# Recalculate the global mean using the training set.
```

```
mu <- mean(train_set$rating)
```

```
# Movie effect
```

```
b_i <- train_set %>%
  group_by(movieId) %>%
  summarize(
    b_i = sum(rating - mu)/(n() + lambda),
    .groups = "drop"
  )
```

```

# User effect
b_u <- train_set %>%
  left_join(b_i, by = "movieId") %>%
  group_by(userId) %>%
  summarize(
    b_u = sum(rating - mu - b_i)/(n() + lambda),
    .groups = "drop"
  )

# Genre effect
b_g <- train_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  group_by(genre) %>%
  summarize(
    b_g = sum(rating - mu - b_i - b_u)/(n() + lambda),
    .groups = "drop"
  )

# Year effect
b_y <- train_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  group_by(year) %>%
  summarize(
    b_y = sum(rating - mu - b_i - b_u - b_g)/(n() + lambda),
    .groups = "drop"
  )

# Generate validation predictions using all learned effects.
pred_final <- validation_set %>%
  left_join(b_i, by = "movieId") %>%
  left_join(b_u, by = "userId") %>%
  left_join(b_g, by = "genre") %>%
  left_join(b_y, by = "year") %>%
  mutate(
    across(starts_with("b_"), ~ifelse(is.na(.x), 0, .x)),
    pred = mu + b_i + b_u + b_g + b_y
  )

# Evaluate the feature-enhanced model on the validation set.
rmse_final <- RMSE(validation_set$rating, pred_final$pred)

# Summarize RMSE across all models to show incremental performance improvements.
final_results <- tibble(
  model = c(
    "Baseline",
    "Movie Effect",
    "Movie + User Effect",
    "Regularised Model (Movie + User Effect)",
    "Final Model (additional features)"
  ),

```

```

RMSE = c(
  baseline_rmse,
  rmse_movie,
  rmse_user,
  best_rmse,
  rmse_final
)
)

# Display the full model comparison table.
knitr::kable(
  final_results,
  caption = "(\\#tab:final) RMSE comparison across all models",
  escape = TRUE,
  position = "H"
)

```

Table 7: RMSE comparison across all models

model	RMSE
Baseline	1.0603099
Movie Effect	0.9436226
Movie + User Effect	0.8663835
Regularised Model (Movie + User Effect)	0.8658114
Final Model (additional features)	0.8654575

The model was trained using the best lambda value found earlier, where λ is 5, and then evaluated on the validation set.

The improvement from adding genre and release year was modest, but it was still measurable (Table 7). This result suggests that these features contain some extra predictive information, although most of the model's performance still comes from the movie and user effects.

This was an important result because it showed that adding more features does not necessarily lead to a large improvement. In this case, genre and release year helped slightly, but the gain was much smaller than the earlier improvements from adding movie and user effects. For that reason, the feature-enhanced model was treated as the final version, but with the understanding that its main strength still comes from the regularised bias terms rather than the added features.

4 Results

4.1 Final model evaluation on final hold-out test set

After choosing the final model using the validation set, the final model was refit on the full `edx` dataset before generating predictions for `final_holdout_test`. This allowed the final model to use all available training data while still keeping the final hold-out set separate.

The `final_holdout_test` set was used exactly once, after the final model structure and lambda value had been selected using the validation set. It was not used during model development, feature selection, or regularisation tuning. This made it a useful final check of how well the model performed on unseen data.

The final model incorporates:

- the global average rating
- regularised movie effects
- regularised user effects
- regularised genre effects
- regularised release year effects

The RMSE on the final hold-out test set was:

```
## [1] "Final RMSE: 0.864434422241387"
```

This means the model's predictions were off by about 0.864 rating points, on average, using the RMSE metric. The result is slightly better than the required target of 0.86490, so the model meets the project goal.

The final test RMSE was also very close to the validation RMSE of 0.8654575. This was encouraging because it suggested that the model had not simply been fitted to the validation set. Instead, it appeared to generalise well to new data.

Overall, the hold-out test result confirmed that the final model captured the main patterns in the dataset. The regularised movie and user effects accounted for most of the predictive power, while genre and release year provided a small additional improvement.

4.2 Discussion of results

The final model performed much better than the baseline model, showing that ratings could not be explained well by the global average alone. The largest improvements came from accounting for systematic differences between movies and users.

Some movies were consistently rated above or below average, while users also differed in how generously or critically they used the rating scale. Including both sources of variation allowed the model to capture these patterns more accurately.

Regularisation gave a smaller improvement, but it was still important. It helped control the effect of movies or users with only a small number of ratings, where the estimated effects were more likely to be unreliable.

Adding genre and release year produced a small further improvement in RMSE. This suggests that these features contained useful information, but most of the predictive power still came from the regularised user and movie effects.

Overall, the results show that the most important part of the model was the regularised movie and user effects. The extra features helped, but mainly as small refinements rather than major drivers of performance.

4.3 Future work

One clear next step would be to use matrix factorisation. The models in this report mainly capture average differences between users and movies, but matrix factorisation could capture more detailed patterns in user preferences. For example, it may identify groups of users who tend to like similar types of films, even if those patterns are not directly visible from genre alone.

Another improvement would be to use all listed genres for each movie instead of only the first genre. In this report, a movie such as *Action/Adventure/Sci-Fi* was treated only as *Action*, which simplified the modelling process but lost some information. A multi-genre approach may give a more accurate representation of each movie.

It may also be useful to include time-based effects. User preferences can change over time, and rating behaviour may also shift depending on when the rating was submitted. Adding features based on rating date could help capture these changes.

Finally, more flexible models such as gradient boosting or neural networks could be tested. These approaches may be able to capture non-linear relationships that the current bias-based model does not include. However, they would also add complexity, so their performance would need to be compared carefully against the simpler regularised model used here.

Overall, the current model performs well for a relatively simple approach, but these extensions could potentially reduce RMSE further by capturing more complex relationships in the data.

5 Conclusion

In this project, a series of models were built to predict movie ratings in the MovieLens dataset. The modelling process started with a simple global average model and then gradually added movie effects, user effects, regularisation, genre, and release year.

The exploratory analysis helped guide these choices. For example, the uneven number of ratings per movie and per user showed why regularisation was needed, while the differences in average rating by genre and release year suggested that these features were worth testing.

The final model achieved an RMSE of 0.8644344 on the hold-out test set, which met the required target. This result showed that the model was able to make reasonably accurate predictions on data that had not been used during training or validation.

The strongest improvements came from modelling systematic differences between movies and users. Genre and release year provided a small additional gain, but their effect was secondary.

Overall, this project showed that a relatively simple regularised bias model can perform well on a recommendation problem. At the same time, the model is still limited because it does not capture more complex relationships between users and movies. More advanced methods, such as matrix factorisation or time-based modelling, could be explored in future work.

6 Acknowledgement of generative AI use

ChatGPT was used sparingly for wording suggestions, debugging support, and review. All final code, modelling decisions, interpretation, and conclusions were reviewed, tested, adapted, and written by me.

7 References

GroupLens Research. (2009). MovieLens 10M Dataset. <https://grouplens.org/datasets/movielens/10m/>

8 Appendix - Environment

```
print("Operating System:")
```

```
## [1] "Operating System:"
```

```
version
```

```
##  
## platform      x86_64-w64-mingw32  
## arch          x86_64  
## os            mingw32  
## crt           ucrt  
## system        x86_64, mingw32  
## status  
## major         4  
## minor         5.3  
## year          2026  
## month         03  
## day           11  
## svn rev       89597  
## language      R  
## version.string R version 4.5.3 (2026-03-11 ucrt)  
## nickname      Reassured Reassurer
```